Data Structures
Laboratory Sheet No. 3
Engineering - Instituto Superior de Engenharia de Coimbra

1 - Build a method that takes as a parameter a generic array, as well as a generic element to look for, and returns as a result indicating whether the element is found more than once in the array. The element must be identified by comparison by reference.

2- Build a method that takes as a parameter a generic array, as well as a generic element to look comparable to the elements of the array, and returns the result indicating whether the element is found more than once in the array. The element shall be identified by the compareTo method.

3 - Build a figure / Rectangle hierarchy, in which Figure implements the interface Comparable <Figure>. The comparison between the two figures is made by its area.

a) Construct a method which receives two figures and returns the result of the comparison.

b) Build a method that receives a rectangle and a second object with which the rectangle can be compared and returns the result of the comparison. Check that the method works properly when the second object is any of the following types: Figure Rectangle an X class that is Comparable <Rectangle> a class Y to be Comparable <Figure>, and class Z to be Comparable < Object> (classes X, Y and Z are neither related to the class Figure nor class Rectangle)

c) Build a method that takes an object of any type and a second one with which the first is comparable.

4 - Build a static method that takes two parameters, the first being an array. The method should indicate whether there is a value in the array that iss greater than the second parameter.
Example:

```
Integer m [] = {3,2,6,3};
N String [] = {"Ada", "Albino"};
System.October.println (search (m, 2)); // true
System.October.println (search (n,"Francisco")); // false
```

5 - Build a class representing a bidimensional point, which stores numerical coordinates using generics appropriately. The following code illustrates the kind of operations that should be possible:

```
Point <Integer, Integer> w = new Point<> (3,4);
Point <Number, Number> x = new Point <> (0,0);
System.out.println (w); // outputs (3,4)
System.out.println (x); // prints (0,0)
x.copy (w);
System.out.println (x); // outputs (3,4)
Point <String, Integer> error =new Point <> ( "hello", 3);
     // compilation error
```